

Chapter 9: Population Methods

Algorithms for Optimization, 2nd Edition (2026)

Kochenderfer & Wheeler

Adapted for Lecture Slides

2026

Chapter Outline

- 1 Population Iteration
- 2 Genetic Algorithms
- 3 Differential Evolution
- 4 Particle Swarm Optimization
- 5 Firefly Algorithm
- 6 Cuckoo Search
- 7 Hybrid Methods
- 8 Summary & Exercises

Motivation: Why Populations?

Key Idea

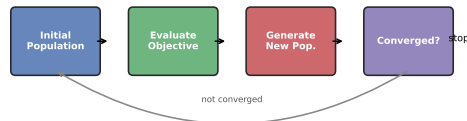
Previous chapters focused on methods where a *single design point* is iterated toward a minimum.

Population methods maintain a *collection* of candidate designs $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$ and evolve them collectively.

Advantages:

- Information at different points can be *shared*
- Better escape from *local minima*
- Naturally *parallelisable*
- Population diversity supports global exploration

Population Iteration Loop



The basic population loop: initialise, evaluate, generate new population, check convergence.

Population Iteration: Algorithm Template

General Population Method (Alg. 9.1)

- 1 **Initialise** population $\mathcal{P} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$, typically by sampling uniformly in the design space.
- 2 For $k = 1, 2, \dots, k_{\max}$:
 - a Evaluate $f(\mathbf{x}^{(i)})$ for all individuals.
 - b **Generate** a new population using a method-specific rule (selection, mutation, etc.).
- 3 Return $\mathbf{x}^* = \arg \min_i f(\mathbf{x}^{(i)})$.

```
1 import numpy as np
2
3 def population_method(f, bounds, pop_size=50, k_max=100, rng=None):
4     """Generic population loop. Subclass and override 'step'."""
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     population = rng.uniform(lo, hi, (pop_size, len(lo)))
8     for _ in range(k_max):
9         y = np.array([f(x) for x in population])
10        population = step(population, y, rng)    # method-specific
11    y = np.array([f(x) for x in population])
12    return population[np.argmin(y)]
```

Listing 1: Population method skeleton (Python)

Genetic Algorithms: Overview

Biological Inspiration (Alg. 9.2)

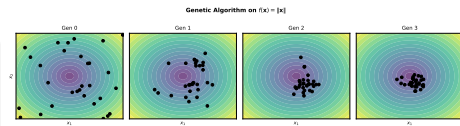
Inspired by natural evolution. Each individual is a *chromosome* (a design vector $\mathbf{x} \in \mathbb{R}^n$). Each generation applies three operators:

- 1 **Selection** — choose parents based on fitness
- 2 **Crossover** — combine parent chromosomes
- 3 **Mutation** — randomly perturb offspring

Fitness of individual i :

$$\text{fitness}_i = \max\{f(\mathbf{x}^{(1)}), \dots, f(\mathbf{x}^{(m)})\} - f(\mathbf{x}^{(i)})$$

Higher fitness $\Rightarrow$ more likely to be selected as a parent.



Four generations of a genetic algorithm optimising $f(\mathbf{x}) = \|\mathbf{x}\|$ (populations converging toward the origin).

9.2.1 Selection

Goal: Choose parent pairs that produce fit offspring.

Truncation Selection

Keep only the top k individuals (by objective value).

Sample pairs uniformly from this elite set.

Simple, fast; may reduce diversity.

Tournament Selection

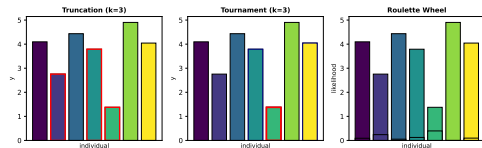
For each parent slot: sample k individuals at random, pick the one with the best objective value.

Controllable selection pressure.

Roulette-Wheel (Fitness-Proportional) Selection

Assign probability proportional to fitness:

$$P(\text{select } i) = \frac{\text{fitness}_i}{\sum \text{fitness}_i}$$



Left: Truncation selects top 3 (red border). **Centre:** Tournament selects winner of a random subset.

Right: Roulette wheel — bar heights show selection likelihood.

Selection: Python Implementation

```
1 import numpy as np
2
3 def truncation_select(y, k, rng):
4     """Return list of parent-index pairs from top-k individuals."""
5     parents = np.argsort(y)[:k]           # k lowest objective values
6     pairs = rng.choice(parents, (len(y), 2))
7     return pairs
8
9 def tournament_select(y, k, rng):
10    """Tournament selection: k random competitors per parent slot."""
11    def pick_one():
12        contenders = rng.choice(len(y), k, replace=False)
13        return contenders[np.argmin(y[contenders])]
14    return [(pick_one(), pick_one()) for _ in range(len(y))]
15
16 def roulette_select(y, rng):
17    """Fitness-proportional (roulette wheel) selection."""
18    fitness = np.max(y) - y               # invert: lower y => higher fitness
19    fitness = np.maximum(fitness, 0)
20    if fitness.sum() == 0:
21        fitness = np.ones_like(fitness)
22    probs = fitness / fitness.sum()
23    idx = rng.choice(len(y), (len(y), 2), p=probs)
24    return idx
```

Listing 2: Three selection methods (Python / NumPy)

9.2.2 Crossover

Goal: Combine the genetic material of two parents to create offspring chromosomes.

Single-Point Crossover

One crossover point i drawn uniformly. Child inherits $\mathbf{x}_a[1:i]$ then $\mathbf{x}_b[i+1:n]$.

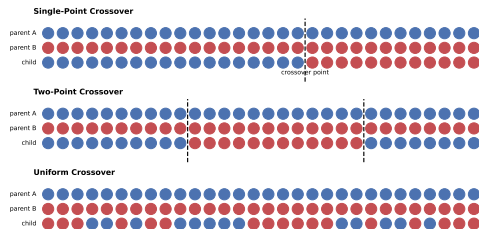
Two-Point Crossover

Two crossover points $i < j$. Child: $\mathbf{x}_a[1:i]$, $\mathbf{x}_b[i+1:j]$, $\mathbf{x}_a[j+1:n]$.

Uniform Crossover

Each gene inherited from parent A with probability p and from parent B with probability $1 - p$. Typically $p = 0.5$.

Interpolation Crossover (Eq. 9.1)



Blue = parent A genes; red = parent B genes.

Top: single-point. **Centre:** two-point. **Bottom:** uniform.

Crossover: Python Implementation

```
1 import numpy as np
2
3 def single_point_crossover(a, b, rng):
4     """Single-point crossover of two real-valued vectors."""
5     i = rng.integers(1, len(a))          # crossover point in [1, n-1]
6     return np.concatenate([a[:i], b[i:]])
7
8 def two_point_crossover(a, b, rng):
9     n = len(a)
10    i, j = sorted(rng.choice(n, 2, replace=False))
11    return np.concatenate([a[:i], b[i:j], a[j:]])
12
13 def uniform_crossover(a, b, p, rng):
14     """Each gene from parent a with prob p, else from b."""
15     mask = rng.random(len(a)) < p
16     return np.where(mask, a, b)
17
18 def interpolation_crossover(a, b, lam=0.5):
19     """Convex combination of two parents."""
20     return (1 - lam) * a + lam * b
```

Listing 3: Crossover operators (Python / NumPy)

9.2.3 Mutation

Why Mutation?

Without mutation, traits absent from the initial population can *never* appear. Crossover alone cannot escape local saturation. Mutation enables *exploration*.

Distribution Mutation (Alg. 9.5):

- Mutation rate: $\lambda \in (0, 1)$, typically $\lambda = 1/m$ where m = chromosome length.
- Each gene v mutated with probability λ :

$$v \leftarrow v + \varepsilon, \quad \varepsilon \sim \mathcal{D}$$

- **Gaussian mutation:** $\mathcal{D} = \mathcal{N}(0, \sigma^2)$
- Binary chromosomes: use *bit-flip* mutation.

Typical: $\lambda = 1/m$, yielding on average *one* mutation per child chromosome.

Gaussian Mutation (Python)

```
def gaussian_mutate(child, sigma,
                    lam=None, rng=None):
    rng = rng or np.random.default_rng()
    m = len(child)
    lam = lam or 1.0 / m
    mask = rng.random(m) < lam
    noise = rng.normal(0, sigma, m)
    return child + mask * noise
```

$\lambda = 1/m$ ensures on average exactly one gene is perturbed per offspring, balancing exploration and exploitation.

Genetic Algorithm: Full Python Example

```
1 import numpy as np
2
3 def genetic_algorithm(f, bounds, pop_size=100, k_max=10,
4                       k_trunc=10, sigma=0.5, rng=None):
5     rng = rng or np.random.default_rng(0)
6     lo, hi = np.array(bounds).T
7     n = len(lo)
8     # Initialise
9     pop = rng.uniform(lo, hi, (pop_size, n))
10    for _ in range(k_max):
11        y = np.array([f(x) for x in pop])
12        # Truncation selection: keep top k_trunc
13        parents = pop[np.argsort(y)[:k_trunc]]
14        new_pop = []
15        while len(new_pop) < pop_size:
16            a_idx, b_idx = rng.choice(k_trunc, 2, replace=False)
17            # Single-point crossover
18            cp = rng.integers(1, n)
19            child = np.concatenate([parents[a_idx, :cp],
20                                   parents[b_idx, cp:]])
21            # Gaussian mutation (rate = 1/n)
22            mask = rng.random(n) < (1.0 / n)
23            child += mask * rng.normal(0, sigma, n)
24            child = np.clip(child, lo, hi)
25            new_pop.append(child)
26        pop = np.array(new_pop)
27    y = np.array([f(x) for x in pop])
```

Genetic Algorithm: Complexity & Notes

Per-Generation Complexity

Step	Cost
Evaluate	$O(m \cdot T_f)$
Sort/select	$O(m \log m)$
Crossover	$O(m \cdot n)$
Mutation	$O(m \cdot n)$
Total/gen	$O(m(T_f + n \log m))$

m = population size, n = dimension, T_f = cost of f .

Running Example (from book)

Minimise $f(\mathbf{x}) = \|\mathbf{x}\|$ on $[-3, 3]^2$:

$m = 100$, $k_{\max} = 10$, $k_{\text{trunc}} = 10$, $\sigma = 0.5$

Result: $\mathbf{x} \approx [-0.0351, 0.0376]$

Design choices:

- **Selection pressure:** large k (truncation) $\Rightarrow$ faster convergence but less diversity.
- **Mutation rate** $\lambda = 1/m$ is a good default for real-valued problems; increase if stuck in local min.
- **Elitism:** always carry over the current best individual unchanged.
- **Population size:** larger m explores better but costs more evaluations per generation.

Differential Evolution (DE)

Core Idea (Alg. 9.6)

DE perturbs each individual $\mathbf{x}$ using the *difference* of two other randomly-chosen population members.

DE/rand/1/bin Algorithm:

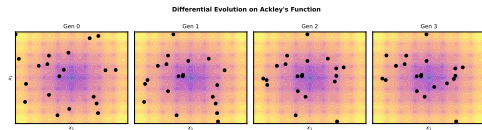
- 1 For each individual $\mathbf{x}$, sample $\mathbf{a}, \mathbf{b}, \mathbf{c}$ from population (all distinct, $\neq \mathbf{x}$).
- 2 Form an *interim design*:

$$\mathbf{z} = \mathbf{a} + w(\mathbf{b} - \mathbf{c})$$

- 3 Apply uniform crossover with probability p :

$$x'_i = \begin{cases} z_i & \text{if } r_i < p \\ x_i & \text{otherwise} \end{cases}$$

(at least one dimension always uses z)

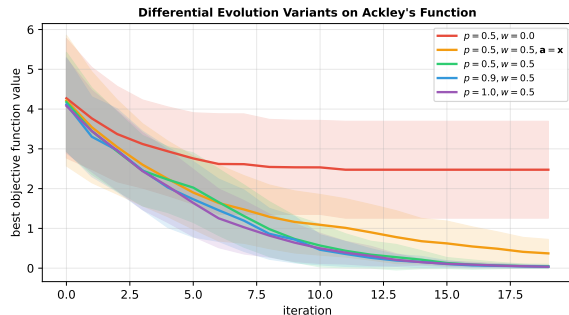


DE on Ackley's function ($p = 0.5$, $w = 0.5$).
Population progressively concentrates near the global minimum.

Differential Evolution: Python

```
1 import numpy as np
2
3 def differential_evolution(f, bounds, pop_size=20, k_max=200,
4                             p=0.5, w=0.5, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     n = len(lo)
8     pop = rng.uniform(lo, hi, (pop_size, n))
9
10    for _ in range(k_max):
11        new_pop = pop.copy()
12        for i in range(pop_size):
13            # Pick 3 distinct indices != i
14            choices = [j for j in range(pop_size) if j != i]
15            a, b, c = pop[rng.choice(choices, 3, replace=False)]
16            # Interim design
17            interim = a + w * (b - c)
18            # Uniform crossover (at least one dimension crossed)
19            mask = rng.random(n) < p
20            if not mask.any():
21                mask[rng.integers(n)] = True
22            child = np.where(mask, interim, pop[i])
23            child = np.clip(child, lo, hi)
24            # Greedy acceptance
25            if f(child) < f(pop[i]):
26                new_pop[i] = child
27    pop = new_pop
```

DE: Parameter Analysis



DE variants on Ackley's function (1000 trials, pop=20, 2D).
Mean $\pm$ 1 std over iterations.

Key Findings (Exercise 9.4)

- $w = 0$ (no differential): stuck — cannot generate new gene values outside current population.
- $a = x$ (perturbation around self): biased toward local exploration, hard to escape local minima.
- Standard $p = 0.5, w = 0.5$: balanced global/local search.
- High crossover $p = 0.9$: nearly as good as guaranteed.
- **Guaranteed crossover** $p = 1.0$: best performance — always fully replaces with interim design.

Role of components:

Particle Swarm Optimization (PSO)

Inspiration

Mimics the social behaviour of bird flocks and fish schools. Each particle has a *position* $\mathbf{x}$, a *velocity* $\mathbf{v}$, and a *personal best* $\mathbf{x}_{\text{best}}$.

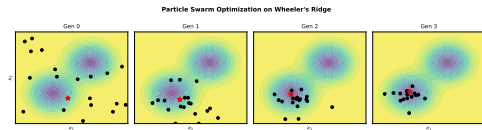
Update Equations (Alg. 9.7):

$$\mathbf{v} \leftarrow w \mathbf{v} + c_1 r_1 \odot (\mathbf{x}_{\text{best}} - \mathbf{x}) + c_2 r_2 \odot (\mathbf{x}^* - \mathbf{x}) \quad (1)$$

$$\mathbf{x} \leftarrow \mathbf{x} + \mathbf{v} \quad (2)$$

Symbol Guide

w	inertia weight
c_1	cognitive coefficient (personal best)
c_2	social coefficient (global best)
r_1, r_2	random vectors $\sim \mathcal{U}[0, 1]^n$
$\mathbf{x}^*$	best position found by any particle



PSO on Wheeler's Ridge. Red star = global best. Particles converge toward two modes.

PSO: Python Implementation

```
1 import numpy as np
2
3 def particle_swarm(f, bounds, n_particles=30, k_max=100,
4                   w=0.1, c1=0.25, c2=2.0, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     n = len(lo)
8     # Initialise positions & velocities
9     pos = rng.uniform(lo, hi, (n_particles, n))
10    vel = rng.normal(0, 0.5, (n_particles, n))
11    x_best = pos.copy()
12    y_best = np.array([f(p) for p in pos])
13    g_best = x_best[np.argmin(y_best)].copy()
14
15    for _ in range(k_max):
16        r1 = rng.random((n_particles, n))
17        r2 = rng.random((n_particles, n))
18        vel = (w * vel
19              + c1 * r1 * (x_best - pos)
20              + c2 * r2 * (g_best - pos))
21        pos = np.clip(pos + vel, lo, hi)
22        y = np.array([f(p) for p in pos])
23        # Update personal bests
24        improved = y < y_best
25        x_best[improved] = pos[improved]
26        y_best[improved] = y[improved]
27        # Update global best
```

PSO: Intuition & Convergence

Three forces on each particle:

- 1 **Inertia** ($w\mathbf{v}$): keeps moving in current direction — prevents premature stopping.
- 2 **Cognitive** ($c_1 r_1 (\mathbf{x}_{\text{best}} - \mathbf{x})$): pulls toward own best — individual memory.
- 3 **Social** ($c_2 r_2 (\mathbf{x}^* - \mathbf{x})$): pulls toward swarm best — collective knowledge.

Convergence control:

- Large $w \Rightarrow$ more exploration
- Large $c_2 \Rightarrow$ faster convergence to swarm best
- If PSO converges too fast to a non-global min: increase n or increase c_1

Complexity

Per iteration: $O(m \cdot T_f + m \cdot n)$

Storage: $O(m \cdot n)$ (positions, velocities, bests)

PSO vs. Genetic Algorithms

	GA	PSO
Memory	no	yes (velocity)
Crossover	yes	no
Mutation	yes	via inertia
Social info	parents	global best

Firefly Algorithm

Inspiration (Yang, 2009)

Fireflies attract each other based on their *brightness* (inversely related to objective value). A less bright firefly moves *toward* a brighter one.

Update Rule (Alg. 9.8):

1 For each pair (i, j) with $f(\mathbf{x}^{(j)}) < f(\mathbf{x}^{(i)})$:

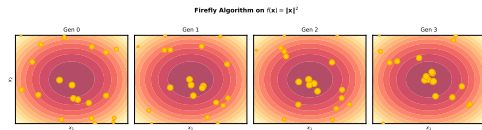
- Compute distance: $r = \|\mathbf{x}^{(i)} - \mathbf{x}^{(j)}\|$
- Attraction: $\beta(r) = \beta_0 e^{-\gamma r^2}$
- Move:

$$\mathbf{x}^{(i)} \leftarrow \mathbf{x}^{(i)} + \beta(r)(\mathbf{x}^{(j)} - \mathbf{x}^{(i)}) + \alpha \epsilon$$

Symbol Guide

β_0 max attractiveness (at $r = 0$)

γ light absorption coefficient



Firefly algorithm on $f(\mathbf{x}) = \|\mathbf{x}\|^2$. Dot size encodes brightness. Fireflies concentrate near origin.

Firefly Algorithm: Python

```
1 import numpy as np
2
3 def firefly(f, bounds, n=20, k_max=100,
4           alpha=0.3, beta0=1.0, gamma=1.0, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     dim = len(lo)
8     pop = rng.uniform(lo, hi, (n, dim))
9
10    for _ in range(k_max):
11        y = np.array([f(p) for p in pop])
12        new_pop = pop.copy()
13        for i in range(n):
14            for j in range(n):
15                if y[j] < y[i]: # j is brighter
16                    r = np.linalg.norm(pop[i] - pop[j])
17                    beta = beta0 * np.exp(-gamma * r**2)
18                    new_pop[i] = (new_pop[i]
19                                + beta * (pop[j] - new_pop[i])
20                                + alpha * rng.normal(0, 1, dim))
21        pop = np.clip(new_pop, lo, hi)
22
23    y = np.array([f(p) for p in pop])
24    return pop[np.argmin(y)]
25
26 # Complexity: O(n^2 * T_f) per iteration --- quadratic in population
27 f = lambda x: np.sum(x**2)
```

Cuckoo Search

Biological Inspiration (Yang & Deb, 2009)

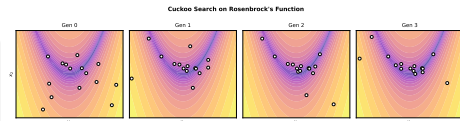
Named after cuckoos that lay eggs in other birds' nests.

- 1 Each cuckoo lays one egg in a randomly chosen nest.
- 2 Best nests (lowest f) carry over to next generation.
- 3 A host bird discovers a foreign egg with probability p_a and abandons it.

Algorithm (Alg. 9.9):

- 1 For each of m_s nests, choose a random nest j and a *Cauchy flight* step $\mathbf{x}' = \mathbf{x}^{(j)} + \text{Cauchy}(r)$; replace j if $f(\mathbf{x}') < f(\mathbf{x}^{(j)})$.
- 2 Sort nests by f .
- 3 Abandon $\lfloor m \cdot p_a \rfloor$ worst nests; replace by uniform random samples.

Parameters: population m , search fraction m_s/m , abandonment probability p_a , Cauchy scale r



Cuckoo search on Rosenbrock's function. Nests (white dots) progressively concentrate near (1, 1).

Cuckoo Search: Python

```
1 import numpy as np
2
3 def cuckoo_search(f, bounds, pop_size=25, k_max=100,
4                   p_a=0.25, m_search_frac=1.0, r=0.01, rng=None):
5     rng = rng or np.random.default_rng()
6     lo, hi = np.array(bounds).T
7     n = len(lo)
8     m_search = max(1, int(m_search_frac * pop_size))
9     m_abandon = max(1, int(p_a * pop_size))
10
11     # Initialise: list of (x, y) sorted by y
12     pop = [(x, f(x)) for x in rng.uniform(lo, hi, (pop_size, n))]
13     pop.sort(key=lambda t: t[1])
14
15     for _ in range(k_max):
16         # Levy-flight step (approximated by Cauchy)
17         for i in range(m_search):
18             j = rng.integers(pop_size)
19             step = rng.standard_cauchy(n) * r * (hi - lo)
20             x_new = np.clip(pop[j][0] + step, lo, hi)
21             y_new = f(x_new)
22             if y_new < pop[i][1]:
23                 pop[i] = (x_new, y_new)
24         # Sort and abandon worst nests
25         pop.sort(key=lambda t: t[1])
26         for i in range(pop_size - m_abandon, pop_size):
27             x_new = rng.uniform(lo, hi, n)
```

Cuckoo Search: Key Points

Levy / Cauchy Flights:

- Cauchy distribution has heavier tails than Gaussian
- Enables occasional *long jumps* — better global exploration
- Short steps dominate most iterations (local refinement)
- Balance: scale parameter r controls step magnitude

Nest Abandonment as Restart

Abandoning p_a fraction of the worst nests with new random samples is equivalent to a *partial restart*, helping escape local minima while preserving elite solutions.

Algorithm Complexity

Step	Cost
m_s Levy flights	$O(m_s \cdot T_f)$
Sort nests	$O(m \log m)$
Abandon/replace	$O(m_a \cdot T_f)$
Total/iter	$O(m \cdot T_f)$

Parameters Summary

Param	Meaning	Typical
m	population	15–50
p_a	abandon rate	0.25
r	Cauchy scale	0.01

9.7 Hybrid Methods

Motivation

Pure population methods excel globally but can be slow to *refine* near a minimum. Pure local search is fast but gets trapped. **Hybrid methods** combine both.

Two philosophies for embedding local search:

Lamarckian Learning

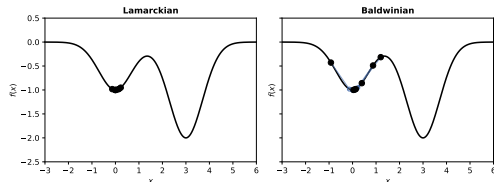
Perform local search on each individual; the individual *physically moves* to the local minimum it found.

Pro: fast convergence near local min.

Con: population clusters prematurely — may miss global optimum.

Baldwinian Learning

Perform local search to *evaluate* each individual; but the



Left (Lamarckian): individuals moved to local min near $x = 0$ — miss global min near $x = 3$.

Right (Baldwinian): original positions preserved (filled dots); local-search evaluations shown as open circles with arrows.

Baldwinian Hybrid: Python Example

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def baldwinian_ga(f, bounds, pop_size=30, k_max=50,
5                  k_trunc=8, sigma=0.3, rng=None):
6     rng = rng or np.random.default_rng()
7     lo, hi = np.array(bounds).T
8     n = len(lo)
9     pop = rng.uniform(lo, hi, (pop_size, n))
10
11     for _ in range(k_max):
12         # Baldwinian: use local search value, keep original position
13         def local_val(x):
14             res = minimize(f, x, method='Nelder-Mead',
15                           options={'maxiter': 20, 'xatol': 1e-4})
16             return res.fun # use improved value for selection
17
18         y = np.array([local_val(x) for x in pop])
19
20         # Truncation selection + crossover + mutation (standard GA)
21         parents = pop[np.argsort(y)[:k_trunc]]
22         new_pop = []
23         while len(new_pop) < pop_size:
24             a, b = parents[rng.choice(k_trunc, 2, replace=False)]
25             cp = rng.integers(1, n)
26             child = np.concatenate([a[:cp], b[cp:]])
27             child += rng.normal(0, sigma, n) * (rng.random(n) < 1/n)
```

Lamarckian vs. Baldwinian: Comparison

Example 9.2 (book)

$f(x) = -e^{-x^2} - 2e^{-(x-3)^2}$, population near $x = 0$.

Lamarckian: individuals move to local min near $x = 0$; future search unlikely to discover global optimum at $x \approx 3$.

Baldwinian: individuals stay at original positions; selection uses local-search values, but diversity preserved.

Method Comparison Table

Property	Lamarckian	Baldwinian
Exploits local	Yes (strongly)	Yes (weakly)
Preserves diversity	No	Yes
Convergence speed	Fast locally	Moderate
Global reach	Low (premature)	Higher
Extra cost	$O(T_{\text{local}})$	$O(T_{\text{local}})$

Rule of thumb: use Baldwinian when the landscape has many local minima and you suspect the global minimum is far from the initial population.

Chapter 9 Summary

Population Methods at a glance:

- Maintain m candidates; evolve collectively.
- Enable *information sharing* across design space.
- Generally *derivative-free*; work on black-box f .

Genetic Algorithms (GA):

- Selection + Crossover + Mutation
- Flexible selection (truncation, tournament, roulette)
- Many crossover schemes for continuous/discrete x

Differential Evolution (DE):

- Uses population *differences* for perturbation
- Greedy acceptance; parameter-efficient

Particle Swarm Optimization (PSO):

- Velocity memory; cognitive + social forces
- Fast convergence; simple implementation

Firefly Algorithm:

- Pairwise attraction by brightness; $O(m^2)$

Cuckoo Search:

- Levy/Cauchy flights for long-range jumps
- Nest abandonment as partial restart

Hybrid Methods:

- Lamarckian: move individuals to local min
- Baldwinian: score by local min, keep position
- Baldwinian preferred for diversity

Method Comparison I

Algorithm Complexity Summary

Method	Per-iteration cost	Key params	Best for
Genetic Algorithm	$O(m(T_f + n \log m))$	m, k, λ, σ	discrete + continuous
Differential Evol.	$O(m \cdot T_f)$	m, p, w	continuous, multi-modal
PSO	$O(m \cdot T_f)$	m, w, c_1, c_2	continuous, fast conv.
Firefly	$O(m^2 T_f)$	$m, \alpha, \beta_0, \gamma$	multi-modal, small m
Cuckoo Search	$O(m \cdot T_f)$	m, p_a, r	continuous, rugged

Practical Guidance

- **Unknown landscape:** start with PSO or DE — few parameters, fast, robust.
- **Mixed integer / combinatorial:** GA with appropriate crossover and mutation.
- **Highly multi-modal:** cuckoo search (long Cauchy flights) or firefly (pairwise attraction).
- **Need local refinement:** wrap any method with Baldwinian hybrid.
- **Budget constraint:** population size m is the primary knob — larger m costs more per iteration but improves coverage.

Selected Exercises

9.1 *What is the motivation behind the selection operation in genetic algorithms?*

Solution: to bias survival toward fitter individuals (lower objective), driving the population toward promising regions.

9.2 *Why does mutation play a fundamental role? How should λ be set if a better optimum is suspected?*

Solution: Mutation drives exploration. Without it, the GA can only recombine existing traits. If a better solution exists further away, *increase* λ .

9.3 *PSO converges to a non-global minimum. How to adjust parameters?*

Solution: increase population size m or the personal-best coefficient c_1 to reduce premature social pull.

9.4 *Compare five DE variants on Ackley's function.*

Solution: $p = 1.0$, $w = 0.5$ performs best (guaranteed crossover). $w = 0$ performs worst (no new gene values can be generated outside existing population).

References

- M. J. Kochenderfer & T. A. Wheeler, *Algorithms for Optimization*, 2nd ed., MIT Press, 2026. Chapter 9.
- J. H. Holland, *Adaptation in Natural and Artificial Systems*, MIT Press, 1992. (Genetic Algorithms)
- R. Storn & K. Price, “Differential Evolution – a simple and efficient heuristic for global optimization,” *J. Global Optim.*, 11(4):341–359, 1997.
- J. Kennedy & R. Eberhart, “Particle Swarm Optimization,” *Proc. ICNN*, pp. 1942–1948, 1995.
- X.-S. Yang, “Firefly Algorithms for Multimodal Optimization,” *Stochastic Algorithms: Foundations and Applications*, LNCS 5792, 2009.
- X.-S. Yang & S. Deb, “Cuckoo Search via Levy Flights,” *World Congress on Nature & Biologically Inspired Computing*, 2009.